

Moving Beyond Linearity

Polynomials, Step Functions, Splines, Local
Regression, and GAMs

Dr. Rajesh Kumar Mundotiya

Assistant Professor, Department of Computer Science
and Engineering

Lab In-Charge: MATRA Lab, Indian Institute of
Technology Bhilai



MATRA LAB

Session Roadmap

Recap & Objectives

Mathematical Prerequisites

Polynomial Regression & Step Functions

Basis Functions: A Unifying View

Regression Splines

Smoothing Splines

Local Regression

Generalized Additive Models

Applied View

Summary



SECTION 1

Recap & Objectives

From the previous session

Ridge and lasso control variance by **shrinking coefficients** $\hat{\beta}$ toward zero. Both stay firmly *within* the linear model family: $Y = \beta_0 + \sum_j X_j \beta_j + \varepsilon$; only the size of the coefficients is constrained, not the functional form.

The gap this session fills

Shrinkage cannot fix a model that is wrong in *shape*. What if the true relationship between X and Y simply is not a straight line? This session relaxes **linearity itself** while keeping the regression machinery — and interpretability — intact.

From the previous session

Ridge and lasso control variance by **shrinking coefficients** $\hat{\beta}$ toward zero. Both stay firmly *within* the linear model family: $Y = \beta_0 + \sum_j X_j \beta_j + \varepsilon$; only the size of the coefficients is constrained, not the functional form.

The gap this session fills

Shrinkage cannot fix a model that is wrong in *shape*. What if the true relationship between X and Y simply is not a straight line? This session relaxes **linearity itself** while keeping the regression machinery — and interpretability — intact.

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

Learning Objectives

By the end of this session you will be able to:

1. Explain basis expansion as the single idea unifying polynomials, step functions, and splines.
2. Derive the truncated power basis for a cubic spline and count its degrees of freedom.
3. Contrast regression splines, natural splines, and smoothing splines, and explain the smoothness/knot trade-off.
4. Fit and interpret local regression (LOESS).
5. Extend flexible one-dimensional smoothing to many predictors via generalized additive models (GAMs).

The Mathematics We Need First

1. Basis expansion — the unifying idea

Replace the raw predictor x by fixed transformations $h_1(x), \dots, h_M(x)$ and fit $f(x) = \sum_m \beta_m h_m(x)$ by ordinary least squares. The model is **linear in the coefficients**, however curved it is in x .

2. Piecewise functions & smoothness

A piecewise function uses *different* formulas on intervals of x separated by **knots** $\xi_1 < \dots < \xi_K$. It is C^2 if its value, first, and second derivatives are all continuous at every knot — no visible jumps or corners.

3. Penalized least squares

Minimizing fit error $+$ $\lambda \times$ roughness trades two goals off with one knob λ ; as λ grows, variance falls and bias rises — the same trade-off as ridge regression, now applied to a whole function.

The Mathematics We Need First

1. Basis expansion — the unifying idea

Replace the raw predictor x by fixed transformations $h_1(x), \dots, h_M(x)$ and fit $f(x) = \sum_m \beta_m h_m(x)$ by ordinary least squares. The model is **linear in the coefficients**, however curved it is in x .

2. Piecewise functions & smoothness

A piecewise function uses *different* formulas on intervals of x separated by **knots** $\xi_1 < \dots < \xi_K$. It is C^2 if its value, first, and second derivatives are all continuous at every knot — no visible jumps or corners.

3. Penalized least squares

Minimizing fit error $+$ $\lambda \times$ roughness trades two goals off with one knob λ ; as λ grows, variance falls and bias rises — the same trade-off as ridge regression, now applied to a whole function.

From One Model to Many: The Menu

Method	Main idea
Polynomial regression	Use powers $x, x^2, \dots, x^d$ as basis functions.
Step functions	Partition X into regions, fit a constant in each.
Regression splines	Piecewise low-degree polynomials, joined smoothly at knots.
Smoothing splines	Minimize error plus a roughness penalty — no knot choice.
Local regression	Weighted least squares fit locally around each target point.
Additive models (GAM)	$\sum_j f_j(X_j)$, one smooth function per predictor.

Polynomial Regression: Core Model

Degree- d polynomial

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i. \quad (7.1)$$

- ▶ Non-linear in x , but **linear in the coefficients** $\Rightarrow$ fit by ordinary least squares.
- ▶ Basis functions: $h_1(x) = x$, $h_2(x) = x^2, \dots, h_d(x) = x^d$, so $f(x) = \beta_0 + \sum_m \beta_m h_m(x) = \mathbf{h}(x)^\top \hat{\beta}$.
- ▶ Large d makes the fit unstable, especially near the boundaries of the data — flexibility is applied **globally**.

Standard errors and logistic version

$\text{Var}\{\hat{f}(x_0)\} = \mathbf{h}(x_0)^\top \text{Var}(\hat{\beta}) \mathbf{h}(x_0)$ gives pointwise confidence bands; the same basis expansion inside a logistic link, $\text{logit}\{p(x)\} = \beta_0 + \sum_m \beta_m x^m$, turns it into **polynomial logistic regression** for a binary response.

Polynomial Regression: Core Model

Degree- d polynomial

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i. \quad (7.1)$$

- ▶ Non-linear in x , but **linear in the coefficients** $\Rightarrow$ fit by ordinary least squares.
- ▶ Basis functions: $h_1(x) = x$, $h_2(x) = x^2, \dots, h_d(x) = x^d$, so $f(x) = \beta_0 + \sum_m \beta_m h_m(x) = \mathbf{h}(x)^\top \hat{\beta}$.
- ▶ Large d makes the fit unstable, especially near the boundaries of the data — flexibility is applied **globally**.

Standard errors and logistic version

$\text{Var}\{\hat{f}(x_0)\} = \mathbf{h}(x_0)^\top \text{Var}(\hat{\beta}) \mathbf{h}(x_0)$ gives pointwise confidence bands; the same basis expansion inside a logistic link, $\text{logit}\{p(x)\} = \beta_0 + \sum_m \beta_m x^m$, turns it into **polynomial logistic regression** for a binary response.

Figure: Polynomial Fit and Polynomial Logistic Regression

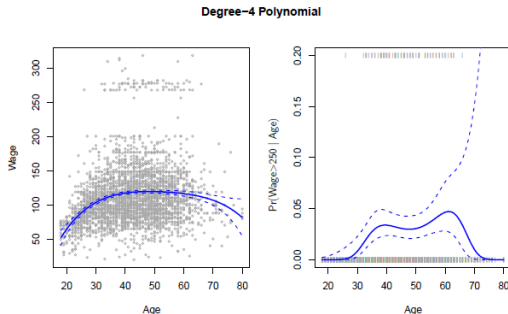


FIGURE 7.1. The **Wage** data. Left: The solid blue curve is a degree-4 polynomial of **wage** (in thousands of dollars) as a function of **age**, fit by least squares. The dashed curves indicate an estimated 95 % confidence interval. Right: We model the binary event **wage**>250 using logistic regression, again with a degree-4 polynomial. The fitted posterior probability of **wage** exceeding \$250,000 is shown in blue, along with an estimated 95 % confidence interval.

Left panel

Wage as a degree-4 polynomial of age: rises, then declines with age. Dashed curves are point-wise $\pm 2SE$ bands.

Right panel

$Y = 1$ if wage > \$250k; logistic regression estimates $\Pr(Y = 1 \mid X = x)$. Bands widen where high earners are rare.

Step Functions

Idea

Cut cutpoints $\xi_1 < \dots < \xi_K$, build indicators $C_k(X) = I(\xi_k \leq X < \xi_{k+1})$, and fit a **piecewise constant** model:

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i) + \varepsilon_i. \quad (7.5)$$

- ▶ $C_0(X)$ is dropped (redundant with the intercept); β_0 = mean response in the first bin; β_j = the shift for bin j relative to it.
- ▶ Converts a continuous predictor into an ordered categorical one.
- ▶ Easy to interpret, but crude if the true trend is smooth — a step function cannot represent gradual change within a bin.

Step Functions

Idea

Cut cutpoints $\xi_1 < \dots < \xi_K$, build indicators $C_k(X) = I(\xi_k \leq X < \xi_{k+1})$, and fit a **piecewise constant** model:

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i) + \varepsilon_i. \quad (7.5)$$

- ▶ $C_0(X)$ is dropped (redundant with the intercept); β_0 = mean response in the first bin; β_j = the shift for bin j relative to it.
- ▶ Converts a continuous predictor into an ordered categorical one.
- ▶ Easy to interpret, but crude if the true trend is smooth — a step function cannot represent gradual change within a bin.

Figure: Step Function Fits

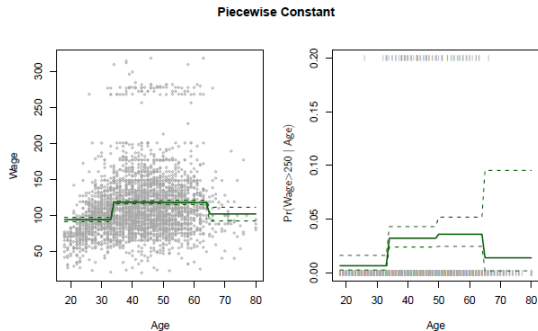


FIGURE 7.2. The **Wage** data. Left: The solid curve displays the fitted value from a least squares regression of **wage** (in thousands of dollars) using step functions of **age**. The dashed curves indicate an estimated 95 % confidence interval. Right: We model the binary event **wage**>250 using logistic regression, again using step functions of **age**. The fitted posterior probability of **wage** exceeding \$250,000 is shown, along with an estimated 95 % confidence interval.

What you see

Left: piecewise-constant wage across age bins. Right: logistic regression on the same bins — log-odds constant within a bin, jumps only at cutpoints.

Limitation

Crude when the true trend is smooth; sensitive to where the cuts are placed.

Basis Functions: The General Framework

Basis expansion model

$$f(x) = \sum_{m=1}^M \beta_m h_m(x), \quad y_i = f(x_i) + \varepsilon_i. \quad (7.7)$$

- ▶ $h_1, \dots, h_M$ are fixed transformations chosen *before* fitting — the model is linear in β , flexible in x .
- ▶ Matrix form: $\mathbf{y} = \mathbf{H}\beta + \varepsilon$, $\hat{f}(x) = \mathbf{h}(x)^\top \hat{\beta}$ — ordinary least squares applies unchanged.

Everything so far is the same trick

Polynomial regression	$h_m(x) = x^m$
Step functions	$h_m(x) = I(\xi_m \leq x < \xi_{m+1})$
Regression splines	piecewise-polynomial basis functions
Fourier / wavelets	periodic / localized basis functions

Basis Functions: The General Framework

Basis expansion model

$$f(x) = \sum_{m=1}^M \beta_m h_m(x), \quad y_i = f(x_i) + \varepsilon_i. \quad (7.7)$$

- ▶ $h_1, \dots, h_M$ are fixed transformations chosen *before* fitting — the model is linear in β , flexible in x .
- ▶ Matrix form: $\mathbf{y} = \mathbf{H}\beta + \varepsilon$, $\hat{f}(x) = \mathbf{h}(x)^\top \hat{\beta}$ — ordinary least squares applies unchanged.

Everything so far is the same trick

Polynomial regression	$h_m(x) = x^m$
Step functions	$h_m(x) = I(\xi_m \leq x < \xi_{m+1})$
Regression splines	piecewise-polynomial basis functions
Fourier / wavelets	periodic / localized basis functions

Regression Splines: Local Flexibility

Main idea

Fit low-degree polynomials over **local regions** of X , separated at **knots** $\xi_1, \dots, \xi_K$, and force the pieces to join smoothly.

- ▶ With K knots there are $K + 1$ regions, each with its own cubic:
$$y_i = \beta_{0j} + \beta_{1j}x_i + \beta_{2j}x_i^2 + \beta_{3j}x_i^3 + \varepsilon_i.$$
- ▶ Unconstrained, the curve can jump at a knot.
- ▶ A **cubic spline** adds continuity of the function, first derivative, *and* second derivative at every knot — no visible corners.

Why splines beat a single global polynomial

They add flexibility *locally*, by inserting knots, instead of raising the polynomial degree *globally*.

Regression Splines: Local Flexibility

Main idea

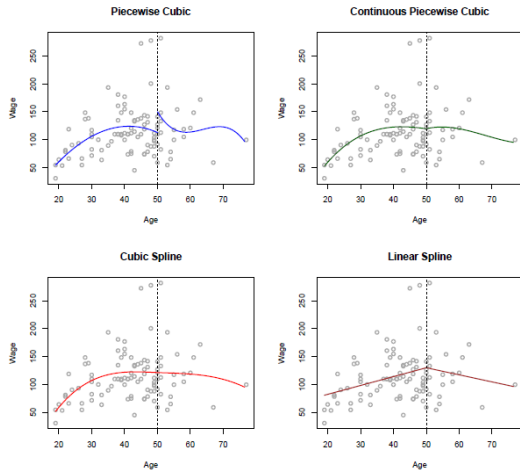
Fit low-degree polynomials over **local regions** of X , separated at **knots** $\xi_1, \dots, \xi_K$, and force the pieces to join smoothly.

- ▶ With K knots there are $K + 1$ regions, each with its own cubic:
$$y_i = \beta_{0j} + \beta_{1j}x_i + \beta_{2j}x_i^2 + \beta_{3j}x_i^3 + \varepsilon_i.$$
- ▶ Unconstrained, the curve can jump at a knot.
- ▶ A **cubic spline** adds continuity of the function, first derivative, *and* second derivative at every knot — no visible corners.

Why splines beat a single global polynomial

They add flexibility *locally*, by inserting knots, instead of raising the polynomial degree *globally*.

Figure: From Discontinuous to Fully Smooth



- ▶ Top-left: unconstrained piecewise cubic — discontinuous.
- ▶ Top-right: continuity fixes the jump but leaves a corner.
- ▶ Bottom-left: cubic spline — continuous value, 1st, 2nd derivative.
- ▶ Bottom-right: linear spline — continuous straight-line pieces.

FIGURE 7.3. Various piecewise polynomials are fit to a subset of the Wage

Degrees of Freedom and the Truncated Power Basis

Counting parameters

$4(K + 1)$ coefficients across $K + 1$ cubic pieces, minus 3 continuity constraints per knot:

$$4(K + 1) - 3K = K + 4.$$

Truncated power basis

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^K \theta_k (x - \xi_k)_+^3 \quad (7.9)$$

with $(x - \xi)_+^3 = (x - \xi)^3$ if $x > \xi$, else 0.

One basis function per knot

Each added $(x - \xi_k)_+^3$ term keeps the function, its first, and its second derivative continuous at ξ_k — a $K + 4$ -column design matrix, still fit by ordinary least squares.

Natural cubic splines

A **natural cubic spline** adds boundary constraints forcing the fit to be *linear* beyond the two boundary knots — it trades a little bias for much lower variance at the edges, where data is sparsest.

Degrees of Freedom and the Truncated Power Basis

Counting parameters

$4(K + 1)$ coefficients across $K + 1$ cubic pieces, minus 3 continuity constraints per knot:

$$4(K + 1) - 3K = K + 4.$$

Truncated power basis

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^K \theta_k (x - \xi_k)_+^3 \quad (7.9)$$

with $(x - \xi)_+^3 = (x - \xi)^3$ if $x > \xi$, else 0.

One basis function per knot

Each added $(x - \xi_k)_+^3$ term keeps the function, its first, and its second derivative continuous at ξ_k — a $K + 4$ -column design matrix, still fit by ordinary least squares.

Natural cubic splines

A **natural cubic spline** adds boundary constraints forcing the fit to be *linear* beyond the two boundary knots — it trades a little bias for much lower variance at the edges, where data is sparsest.

Figure: Cubic Spline vs Natural Cubic Spline

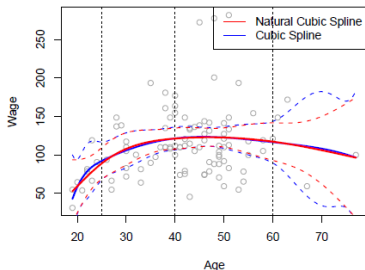


FIGURE 7.4. A cubic spline and a natural cubic spline, with three knots, fit to a subset of the Wage data. The dashed lines denote the knot locations.

What changes

Ordinary cubic splines can swing wildly near the boundaries. The natural cubic spline (constrained to be linear past the boundary knots) is far more stable there, with narrower confidence bands.

Choosing the Number and Location of Knots

- ▶ More knots $\Rightarrow$ more flexibility $\Rightarrow$ lower bias, higher variance.
- ▶ Common practice: place knots at **uniform quantiles** of the predictor, and specify the desired **degrees of freedom** rather than explicit knot locations.
- ▶ **Cross-validation** chooses the degrees of freedom objectively.

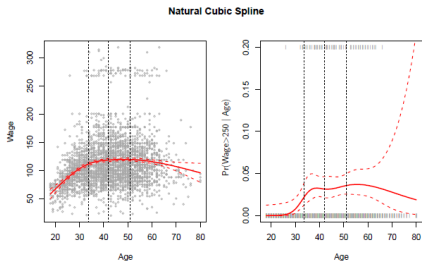


FIGURE 7.5. A natural cubic spline function with four degrees of freedom is fit to the **Wage** data. Left: A spline is fit to **wage** (in thousands of dollars) as a function of **age**. Right: Logistic regression is used to model the binary event **wage** > 250 as a function of **age**. The fitted posterior probability of **wage** exceeding \$250,000 is shown. The dashed lines denote the knot locations.

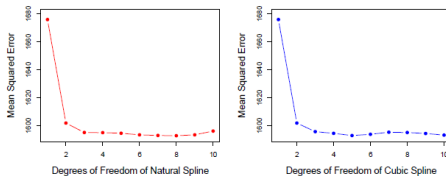


FIGURE 7.6. Ten-fold cross-validated mean squared errors for selecting the degrees of freedom when fitting splines to the **Wage** data. The response is **wage** and the predictor **age**. Left: A natural cubic spline. Right: A cubic spline.

10-fold CV error vs degrees of freedom: error flattens quickly — a small df is enough.

Choosing the Number and Location of Knots

- ▶ More knots $\Rightarrow$ more flexibility $\Rightarrow$ lower bias, higher variance.
- ▶ Common practice: place knots at **uniform quantiles** of the predictor, and specify the desired **degrees of freedom** rather than explicit knot locations.
- ▶ **Cross-validation** chooses the degrees of freedom objectively.

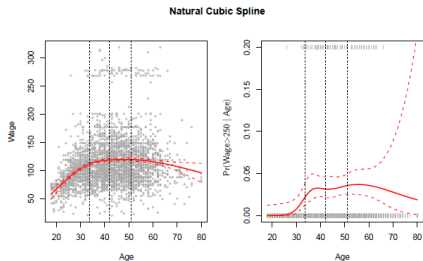


FIGURE 7.5. A natural cubic spline function with four degrees of freedom is fit to the **wage** data. Left: A spline is fit to **wage** (in thousands of dollars) as a function of **age**. Right: Logistic regression is used to model the binary event **wage > 250** as a function of **age**. The fitted posterior probability of **wage** exceeding \$250,000 is shown. The dashed lines denote the knot locations.

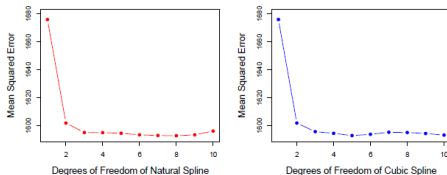


FIGURE 7.6. Ten-fold cross-validated mean squared errors for selecting the degrees of freedom when fitting splines to the **wage** data. The response is **wage** and the predictor **age**. Left: A natural cubic spline. Right: A cubic spline.

10-fold CV error vs degrees of freedom: error flattens quickly — a small df is enough.

Splines vs a High-Degree Polynomial

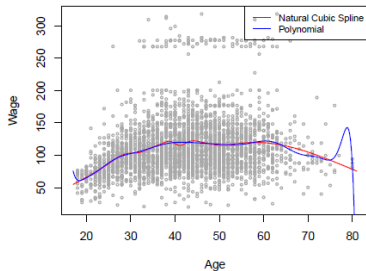


FIGURE 7.7. On the `Wage` data set, a natural cubic spline with 15 degrees of freedom is compared to a degree-15 polynomial. Polynomials can show wild behavior, especially near the tails.

- ▶ Degree-15 polynomial: unstable near the right boundary.
- ▶ Natural cubic spline: stable throughout.

Key reason

Polynomials add flexibility *globally*; splines add it *locally*, at knots — the same lesson as ridge/lasso: control complexity where you need it, not everywhere at once.

Smoothing Splines: No Knots to Choose

Penalized residual sum of squares

$$\min_g \left\{ \sum_{i=1}^N (y_i - g(x_i))^2 + \lambda \int \{g''(t)\}^2 dt \right\}. \quad (7.10)$$

- ▶ First term: fit to the data (RSS).
- ▶ Second term: **roughness penalty** — $g''(t)$ measures curvature, so large curvature is punished.
- ▶ $\lambda \geq 0$ is the single tuning parameter controlling the trade-off.

The remarkable result

The minimizer is always a **natural cubic spline** with knots at *every unique observed* x_i — knot selection disappears entirely; only λ needs to be chosen.

Smoothing Splines: No Knots to Choose

Penalized residual sum of squares

$$\min_g \left\{ \sum_{i=1}^N (y_i - g(x_i))^2 + \lambda \int \{g''(t)\}^2 dt \right\}. \quad (7.10)$$

- ▶ First term: fit to the data (RSS).
- ▶ Second term: **roughness penalty** — $g''(t)$ measures curvature, so large curvature is punished.
- ▶ $\lambda \geq 0$ is the single tuning parameter controlling the trade-off.

The remarkable result

The minimizer is always a **natural cubic spline** with knots at *every unique observed* x_i — knot selection disappears entirely; only λ needs to be chosen.

The Role of λ

$\lambda = 0$

- ▶ No penalty at all.
- ▶ Curve interpolates every training point.
- ▶ Low bias, very high variance.

$\lambda \rightarrow \infty$

- ▶ Curvature forced to zero.
- ▶ Fit approaches the ordinary least-squares straight line.
- ▶ Low variance, potentially high bias.

Effective degrees of freedom

Fitted values are a linear smoother $\hat{\mathbf{g}}_\lambda = \mathbf{S}_\lambda \mathbf{y}$ (7.11), so $\text{df}_\lambda = \text{trace}(\mathbf{S}_\lambda)$ (7.12) measures the *effective* number of fitted parameters — often far fewer than N , despite a knot at every point.

The Role of λ

$\lambda = 0$

- ▶ No penalty at all.
- ▶ Curve interpolates every training point.
- ▶ Low bias, very high variance.

$\lambda \rightarrow \infty$

- ▶ Curvature forced to zero.
- ▶ Fit approaches the ordinary least-squares straight line.
- ▶ Low variance, potentially high bias.

Effective degrees of freedom

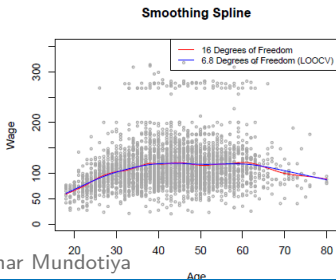
Fitted values are a linear smoother $\hat{\mathbf{g}}_\lambda = \mathbf{S}_\lambda \mathbf{y}$ (7.11), so $\text{df}_\lambda = \text{trace}(\mathbf{S}_\lambda)$ (7.12) measures the *effective* number of fitted parameters — often far fewer than N , despite a knot at every point.

Choosing λ by Leave-One-Out CV

Efficient LOOCV formula for a linear smoother

$$\text{CV}(\lambda) = \sum_{i=1}^N \left(\frac{y_i - \hat{g}_\lambda(x_i)}{1 - (\mathbf{S}_\lambda)_{ii}} \right)^2. \quad (7.13)$$

- ▶ Gives the leave-one-out error *without* refitting N times — exactly the shortcut that makes LOOCV practical for smoothers.
- ▶ Choose the λ that minimizes $\text{CV}(\lambda)$.



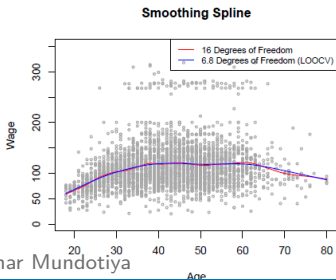
Red: 16 effective df (fixed). Blue: LOOCV-chosen λ , giving 6.8 effective df. The simpler LOOCV fit is nearly identical, so it is preferred.

Choosing λ by Leave-One-Out CV

Efficient LOOCV formula for a linear smoother

$$\text{CV}(\lambda) = \sum_{i=1}^N \left(\frac{y_i - \hat{g}_\lambda(x_i)}{1 - (\mathbf{S}_\lambda)_{ii}} \right)^2. \quad (7.13)$$

- ▶ Gives the leave-one-out error *without* refitting N times — exactly the shortcut that makes LOOCV practical for smoothers.
- ▶ Choose the λ that minimizes $\text{CV}(\lambda)$.



Red: 16 effective df (fixed). Blue: LOOCV-chosen λ , giving 6.8 effective df. The simpler LOOCV fit is nearly identical, so it is preferred.



SECTION 7

Local Regression

Local Regression: Fit Locally, Everywhere

Idea

To estimate $f(x_0)$, fit a *separate* weighted regression using only the observations near x_0 ; nearby points get large weights, far points get zero or tiny weights.

Weighted least squares objective

$$\min_{\beta_0, \beta_1} \sum_{i=1}^N w_i(x_0) (y_i - \beta_0 - \beta_1 x_i)^2, \quad \hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0. \quad (7.14)$$

Memory-based, like KNN

There is no single global $\hat{\beta}$: the fitted model changes with x_0 . The training data must be kept and reused for every new prediction.

Local Regression: Fit Locally, Everywhere

Idea

To estimate $f(x_0)$, fit a *separate* weighted regression using only the observations near x_0 ; nearby points get large weights, far points get zero or tiny weights.

Weighted least squares objective

$$\min_{\beta_0, \beta_1} \sum_{i=1}^N w_i(x_0) (y_i - \beta_0 - \beta_1 x_i)^2, \quad \hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0. \quad (7.14)$$

Memory-based, like KNN

There is no single global $\hat{\beta}$: the fitted model changes with x_0 . The training data must be kept and reused for every new prediction.

Local Regression in Matrix Form

For target x_0 , with $\mathbf{W}(x_0) = \text{diag}\{w_1(x_0), \dots, w_N(x_0)\}$,

$$\hat{\beta}(x_0) = (\mathbf{H}^T \mathbf{W}(x_0) \mathbf{H})^{-1} \mathbf{H}^T \mathbf{W}(x_0) \mathbf{y}, \quad \hat{f}(x_0) = [1 \quad x_0] \hat{\beta}(x_0).$$

Orange line: target x_0 . Orange points: local neighborhood. Yellow bell: the weight kernel. Orange segment: local weighted fit. Orange dot: $\hat{f}(x_0)$.

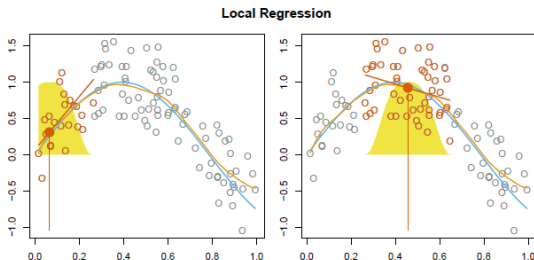


FIGURE 7.9. Local regression illustrated on some simulated data, where the blue curve represents $f(x)$ from which the data were generated, and the light orange curve corresponds to the local regression estimate $\hat{f}(x)$. The orange colored points are local to the target point x_0 , represented by the orange vertical line. The yellow bell-shape superimposed on the plot indicates weights assigned to each point, decreasing to zero with distance from the target point. The fit $\hat{f}(x_0)$ at x_0 is obtained by fitting a weighted linear regression (orange line segment), and using the fitted value at x_0 (orange solid dot) as the estimate $\hat{f}(x_0)$.

Local Regression in Matrix Form

For target x_0 , with $\mathbf{W}(x_0) = \text{diag}\{w_1(x_0), \dots, w_N(x_0)\}$,

$$\hat{\beta}(x_0) = (\mathbf{H}^T \mathbf{W}(x_0) \mathbf{H})^{-1} \mathbf{H}^T \mathbf{W}(x_0) \mathbf{y}, \quad \hat{f}(x_0) = [1 \quad x_0] \hat{\beta}(x_0).$$

Orange line: target x_0 . Orange points: local neighborhood. Yellow bell: the weight kernel. Orange segment: local weighted fit. Orange dot: $\hat{f}(x_0)$.

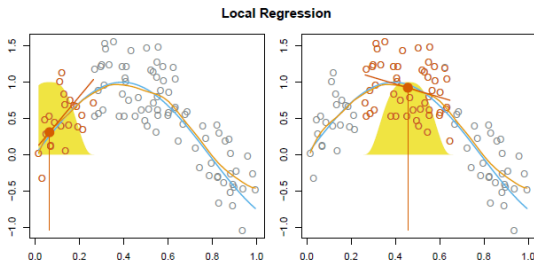


FIGURE 7.9. Local regression illustrated on some simulated data, where the blue curve represents $f(x)$ from which the data were generated, and the light orange curve corresponds to the local regression estimate $\hat{f}(x)$. The orange colored points are local to the target point x_0 , represented by the orange vertical line. The yellow bell-shape superimposed on the plot indicates weights assigned to each point, decreasing to zero with distance from the target point. The fit $\hat{f}(x_0)$ at x_0 is obtained by fitting a weighted linear regression (orange line segment), and using the fitted value at x_0 (orange solid dot) as the estimate $\hat{f}(x_0)$.

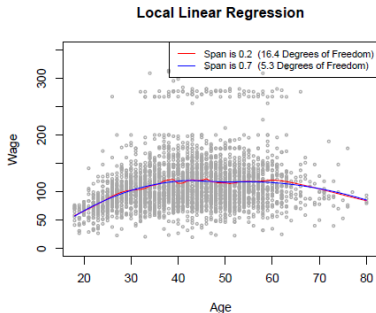


FIGURE 7.10. Local linear fits to the **Wage** data. The span specifies the fraction of the data used to compute the fit at each target point.

- ▶ Span $s = k/N$: fraction of data used at each target point.
- ▶ Small s : very local, flexible, wiggly.
- ▶ Large s : more global, smoother, lower variance.
- ▶ Chosen by cross-validation, exactly as for λ and d .

Curse of dimensionality

Extends to multiple predictors and varying-coefficient models, but local neighborhoods become sparse in high dimensions — the same weakness as KNN.

From One Predictor to Many, Without Losing Interpretability

Additive model

Replace each linear term in multiple regression by a smooth function:

$$y_i = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \varepsilon_i \quad \longrightarrow \quad y_i = \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \varepsilon_i. \quad (7.15)$$

- ▶ Each f_j is estimated with any 1-D smoother (natural spline, smoothing spline, step function, ...).
- ▶ Effects are *added*, so each f_j can be plotted and interpreted separately, holding the others fixed.
- ▶ Additivity avoids the full curse of dimensionality faced by an unrestricted p -dimensional smoother.

From One Predictor to Many, Without Losing Interpretability

Additive model

Replace each linear term in multiple regression by a smooth function:

$$y_i = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \varepsilon_i \quad \longrightarrow \quad y_i = \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \varepsilon_i. \quad (7.15)$$

- ▶ Each f_j is estimated with any 1-D smoother (natural spline, smoothing spline, step function, ...).
- ▶ Effects are *added*, so each f_j can be plotted and interpreted separately, holding the others fixed.
- ▶ Additivity avoids the full curse of dimensionality faced by an unrestricted p -dimensional smoother.

Fitting: Backfitting and Partial Residuals

Backfitting

Update one f_j at a time, holding the others fixed. To update f_j , form the partial residual

$$r_{ij} = y_i - \beta_0 - \sum_{\ell \neq j} f_{\ell}(x_{i\ell}),$$

then smooth r_{ij} against x_{ij} ; repeat until convergence.

Penalized criterion

$$\sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p f_j(x_{ij}) \right)^2 + \sum_{j=1}^p \lambda_j \int \{f_j''(t)\}^2 dt.$$

Identifiability

Each f_j is centered so $\sum_i f_j(x_{ij}) = 0$; the intercept carries the overall mean.

Fitting: Backfitting and Partial Residuals

Backfitting

Update one f_j at a time, holding the others fixed. To update f_j , form the partial residual

$$r_{ij} = y_i - \beta_0 - \sum_{\ell \neq j} f_{\ell}(x_{i\ell}),$$

then smooth r_{ij} against x_{ij} ; repeat until convergence.

Penalized criterion

$$\sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p f_j(x_{ij}) \right)^2 + \sum_{j=1}^p \lambda_j \int \{f_j''(t)\}^2 dt.$$

Identifiability

Each f_j is centered so $\sum_i f_j(x_{ij}) = 0$; the intercept carries the overall mean.

Figure: Additive Model on Wage Data

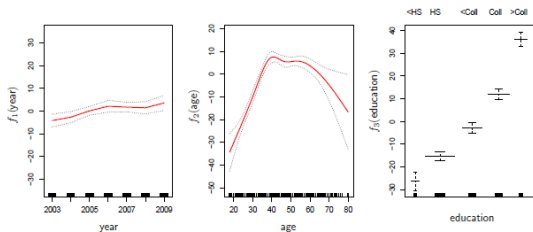


FIGURE 7.11. For the **Wage** data, plots of the relationship between each feature and the response, **wage**, in the fitted model (7.16). Each plot displays the fitted function and pointwise standard errors. The first two functions are natural splines in **year** and **age**, with four and five degrees of freedom, respectively. The third function is a step function, fit to the qualitative variable **education**.

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{educ}) + \varepsilon$$

Holding the others fixed: wage rises slightly with year, peaks in middle age, and increases strongly with education. Each panel is a **partial effect** plot.

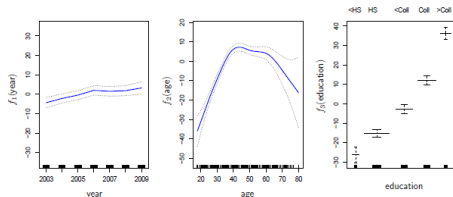


FIGURE 7.12. Details are as in Figure 7.11, but now f_1 and f_2 are smoothing splines with four and five degrees of freedom, respectively.

f_1, f_2 fit by smoothing splines instead of natural splines
— backfitting gives a very similar shape.

Logistic additive model

$$\log\left(\frac{p(\mathbf{x})}{1 - p(\mathbf{x})}\right) = \beta_0 + \sum_{j=1}^p f_j(x_j). \quad (7.18)$$

Same additive idea, now on the log-odds scale —
e.g. $\Pr(\text{wage} > 250 \mid \text{year, age, educ})$ with year
linear, age smoothed, education categorical (7.19).

Advantages

- ▶ Capture non-linear relationships automatically.
- ▶ Often improve accuracy over a linear fit.
- ▶ Each f_j is plottable and interpretable.
- ▶ Smoothness summarized by degrees of freedom per term.

Limitations

- ▶ Restricted to **additive** structure.
- ▶ Important interactions can be missed.
- ▶ Can be added manually via $f_{jk}(X_j, X_k)$, at some cost to interpretability.

In Practice: Fitting Splines in a Few Lines

```
import numpy as np
from patsy import dmatrix
import statsmodels.api as sm

# degree-4 polynomial (np.polyfit, quick)
coefs = np.polyfit(x, y, deg=4)

# natural cubic spline, df=4, via patsy basis
H = dmatrix("cr(x, df=4)", {"x": x})
fit = sm.OLS(y, H).fit()
yhat = fit.predict(H)
```

Same idea, two doors

`np.polyfit` builds the $x, x^2, \dots, x^d$ basis for you; `patsy`'s `cr()` builds a natural-spline basis. Either way the fit is ordinary least squares on a transformed design matrix.

Habit

Pick d , the knots, or λ by cross-validation on held-out data — never by eyeballing the training fit.

Summary & What Comes Next

Key takeaways

- ▶ Basis expansion is the unifying trick: polynomials, step functions, and splines are all ordinary least squares on a transformed x .
- ▶ Splines add flexibility **locally** (knots); smoothing splines replace knot choice with a single penalty λ .
- ▶ Smoothness / complexity is always controlled by *one* tuning parameter (d , df , λ , or span s), chosen by cross-validation.
- ▶ GAMs extend flexible smoothing to many predictors while staying additive and interpretable.

Next session: Trees

All of today's methods fit a *smooth global function*, just with local flexibility built in. **Trees** take a completely different route: instead of a smooth curve, they **partition the predictor space** into rectangular regions and fit a simple constant in each — no smoothness assumption at all.

Summary & What Comes Next

Key takeaways

- ▶ Basis expansion is the unifying trick: polynomials, step functions, and splines are all ordinary least squares on a transformed x .
- ▶ Splines add flexibility **locally** (knots); smoothing splines replace knot choice with a single penalty λ .
- ▶ Smoothness / complexity is always controlled by *one* tuning parameter (d , df , λ , or span s), chosen by cross-validation.
- ▶ GAMs extend flexible smoothing to many predictors while staying additive and interpretable.

Next session: Trees

All of today's methods fit a *smooth global function*, just with local flexibility built in. **Trees** take a completely different route: instead of a smooth curve, they **partition the predictor space** into rectangular regions and fit a simple constant in each — no smoothness assumption at all.

Moving Beyond Linearity

From straight lines to flexible curves, one basis function at a time

Questions & discussion welcome

